

Slide 1 - Titulo

Deteccao de reviews falsas/sinteticas em portugues brasileiro usando CNN 1D

- Estudante: Randerson Sousa de Sá Nunes
- Professor: Prof. Dr. Rafael Torres Anchieta
- Disciplina: Aprendizado Profundo - 3a avaliacao
- Dataset: Fake Reviews PT-BR Dataset
- Projeto: <https://github.com/Altaeir-13/Indago-PT-Detecting-Synthetic-Reviews-in-Brazilian-Portuguese>

Slide 2 - Problema investigado

- Problema: classificar reviews de e-commerce como genuinas ou falsas/sintéticas.
- Relevancia: reviews afetam confiança, reputação de lojas e decisões de compra.
- Tarefa de aprendizado: classificação binária supervisionada de texto.
- Tipo de tarefa: detecção/classificação textual por review individual.
- Modelo de Aprendizado Profundo: CNN 1D em PyTorch.
- Motivação: CNNs capturam padrões locais de palavras com custo menor que modelos Transformer.

Slide 3 - Escopo e limitacoes conceituais

- Unidade de analise: review individual.
- Classe 0: falsa/sintetica, gerada por GPT-2 no dataset.
- Classe 1: genuina.
- A classe falsa e operacional, nao fraude humana comprovada.
- O trabalho nao detecta astroturfing completo.
- Astroturfing exigiria usuario, data, rating, produto, relacoes entre contas e sinais de coordenacao.

Slide 4 - Dataset utilizado

- Nome: Fake Reviews PT-BR Dataset.
- Fonte: <https://github.com/cristianomg10/fake-reviews-ptbr-dataset>
- CSV: true_fake_dataset_top15.csv.
- Base original: Brazilian E-Commerce Public Dataset by Olist.
- Colunas: product_category_name, review_comment_message, label.
- Amostras apos limpeza: 29.988.
- Tipo/formato: texto em portugues brasileiro + categoria + rotulo binario.

![Histograma de tokens](../outputs/figures/review_token_length_hist.png)

![Tamanho por classe](../outputs/figures/review_token_length_by_class.png)

Slide 5 - Exemplos e entrada do modelo

Exemplos curtos do dataset:

- Classe 0, falsa/sintetica: "Demorou um pouco pra chegar, mas fiquei feliz com a compra..."
- Classe 1, genuina: "Sempre comprei neste site e nunca tive qualquer problema, tanto no prazo como no produto."

Forma de entrada textual:

$$X = (x_1, x_2, \dots, x_T)$$

- T é o tamanho da sequência textual.
- x_t representa o token no instante t.
- O texto passa por tokenização, padding/truncamento e embedding treinável.

Slide 6 - EDA do dataset

- Classes quase balanceadas: 15.000 sinteticas e 14.988 genuinas.
- Categorias: 15 grupos de produtos, aproximadamente 2.000 amostras por categoria.
- Ausentes nas colunas canonicas: 0.
- Duplicatas: 683 linhas e 1.537 textos duplicados.
- Media de tamanho: 82,95 caracteres e 13,98 tokens por review.

![Distribuicao de classes](../outputs/figures/class_distribution.png)

![Distribuicao por categoria](../outputs/figures/category_distribution.png)

Slide 7 - Trabalhos relacionados e comparacao

Referencia	Contribuicao	Diferenca para este trabalho
Borges et al. (2025)	Benchmark/dataset de fake reviews PT-BR	Este trabalho foca CNN 1D propria e analise reproduzivel
Kim (2014)	CNNs para classificacao de sentencas	Base conceitual para convolucao em texto
Souza et al. (2020)	BERTimbau para portugues brasileiro	Extensao futura, nao modelo principal

- Este experimento usa review individual, split 70/15/15 e metricas orientadas para a classe 0.
- BERTimbau fica como comparacao futura por ser um modelo contextual pre-treinado.

Slide 8 - Pipeline experimental

1. Carregamento do CSV local.
2. Inferencia das colunas reais.
3. Limpeza minima: espacos/quebras e remocao de textos vazios.
4. EDA: tabelas e figuras em outputs/.
5. Split estratificado: treino, validacao e teste.
6. Baselines: TF-IDF + Regressao Logistica; TF-IDF + SVM Linear.
7. CNN 1D PyTorch com early stopping.
8. Avaliacao no teste e analise qualitativa de erros.

Slide 9 - Modelo implementado: CNN 1D

```
Texto -> tokenizacao -> padding/truncamento -> embedding treinavel  
-> Conv1D -> ReLU -> GlobalMaxPooling1D -> Dropout -> Linear -> logit
```

- Embedding transforma tokens em vetores densos.
- Conv1D captura padroes locais de palavras.
- ReLU adiciona nao linearidade.
- GlobalMaxPooling seleciona os sinais mais fortes.
- Dropout reduz overfitting.
- Linear gera o logit final para BCEWithLogitsLoss.

Slide 10 - Trecho real de código: CNN

Trecho curto da CNN no notebook `notebooks/01_experimento_fake_reviews.ipynb`:

```
self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
self.conv = nn.Conv1d(
    in_channels=embedding_dim,
    out_channels=filters,
    kernel_size=kernel_size,
)
self.relu = nn.ReLU()
self.dropout = nn.Dropout(dropout)
self.classifier = nn.Linear(filters, 1)

features = self.relu(self.conv(embedded.transpose(1, 2)))
pooled = torch.max(features, dim=2).values
```

Slide 11 - Configuracao experimental e metricas

- Linguagem: Python.
- Framework: PyTorch.
- Bibliotecas: pandas, numpy, scikit-learn, matplotlib/seaborn, torch.
- Split: 70% treino, 15% validacao, 15% teste; random_state = 42.
- CNN: vocab_size 20000, max_len 128, embedding_dim 128, filters 128, kernel_size 5.
- Dropout 0.5, lr 0.001, batch_size 64, epochs 30, patience 4.
- Loss: BCEWithLogitsLoss; otimizador: Adam; early stopping por val_loss.

Metricas: acuracia, precisao/recall/F1 da classe 0, F1 macro, AUC-ROC da classe 0 e matriz de confusao.

```
precision_score(y_true, y_pred, pos_label=0)
recall_score(y_true, y_pred, pos_label=0)
f1_score(y_true, y_pred, pos_label=0)
roc_auc_score(y_true == 0, score_label_0)
```

Slide 12 - Resultados quantitativos

Modelo	Accuracy	Precision 0	Recall 0	F1 0	F1 macro	AUC-ROC 0
TF-IDF + Regressao Logistica	0.881085	0.867553	0.899556	0.883264	0.881043	0.949981
TF-IDF + SVM Linear	0.894421	0.886037	0.905333	0.895581	0.894408	0.959195
CNN 1D PyTorch	0.962658	0.963078	0.962222	0.962650	0.962658	0.991639

- CNN: 96,27% de acuracia, 96,27% de F1 macro e 99,16% de AUC-ROC.
- Ganho aproximado sobre o SVM linear: 6,8 pontos percentuais de acuracia.

Slide 13 - Curvas de treino e matriz de confusao

- Menor val_loss observado na epoca 7.
- Treinamento encerrado na epoca 11 por early stopping.
- A matriz de confusao mostra desempenho equilibrado entre as duas classes.

![Curva de loss](../outputs/figures/cnn_loss_curve.png)

![Curva de acuracia](../outputs/figures/cnn_accuracy_curve.png)

![Matriz de confusao CNN](../outputs/figures/confusion_matrix_cnn1d.png)

Slide 14 - Analise de erros

Fonte: outputs/tables/cnn_error_analysis.csv.

- Falso positivo: review genuina classificada como falsa/sintetica.
- Exemplo curto: "Ainda e cedo para avaliar o produto."
- Falso negativo: review falsa/sintetica classificada como genuina.
- Exemplo curto: "Nao gostei dos travesseiros... prazo de entrega e muito longo..."

Possiveis causas:

- Reviews muito curtas.
- Linguagem generica ou padronizada.
- Poucos detalhes concretos.
- Textos sinteticos que imitam padroes reais da Olist.

Slide 15 - Conclusoes e trabalhos futuros

Conclusoes:

- A CNN 1D PyTorch teve o melhor desempenho geral.
- O SVM linear superou a regressao logistica entre os baselines.
- A alta AUC sugere boa separacao entre reviews sinteticas e genuinas no dataset.
- O resultado deve ser interpretado com cautela: nao e deteccao completa de astroturfing.

Trabalhos futuros:

- Comparacao com BERTimbau.
- CNN multi-kernel e validacao cruzada.
- Deduplicacao antes do split.
- Metadados comportamentais, grafos e sinais temporais se houver dados.

Links: projeto GitHub, Fake Reviews PT-BR Dataset e Olist.